HW 4.3 : Reproducible Data Analysis

Mayank Futnani Arupula

Busiest Airports Analysis

This section analyzes the six busiest airports using data from HW #3.4. The goal is to compare airport activity and identify patterns across the airports using both tabular and graphical representations.

Table of Busiest Airports

airport	code	2020	2021	2022	2023	2024	2025
Hartsfield-Jackson Atlanta International Airport	ATL	42,918,685	75,704,760	93,699,630	104,653,451	108,067,766	106,302,208
Dubai International Airport	DXB	NA	29,110,609	66,069,981	86,994,365	92,331,506	95,200,000
Frankfurt Airport	FRA	18,768,601	24,812,849	48,918,482	59,355,389	61,564,957	63,189,666
Los Angeles International Airport	LAX	28,779,527	48,007,284	65,924,298	75,050,875	NA	73,709,594
Heathrow Airport	LHR	22,111,469	NA	61,614,508	79,183,364	83,884,572	84,463,000
Beijing Daxing International Airport	PKX	NA	25,051,012	NA	NA	49,441,029	53,618,949

Table 1: Passenger counts for the six busiest airports from 2020 to 2025.

Graph of Busiest Airports

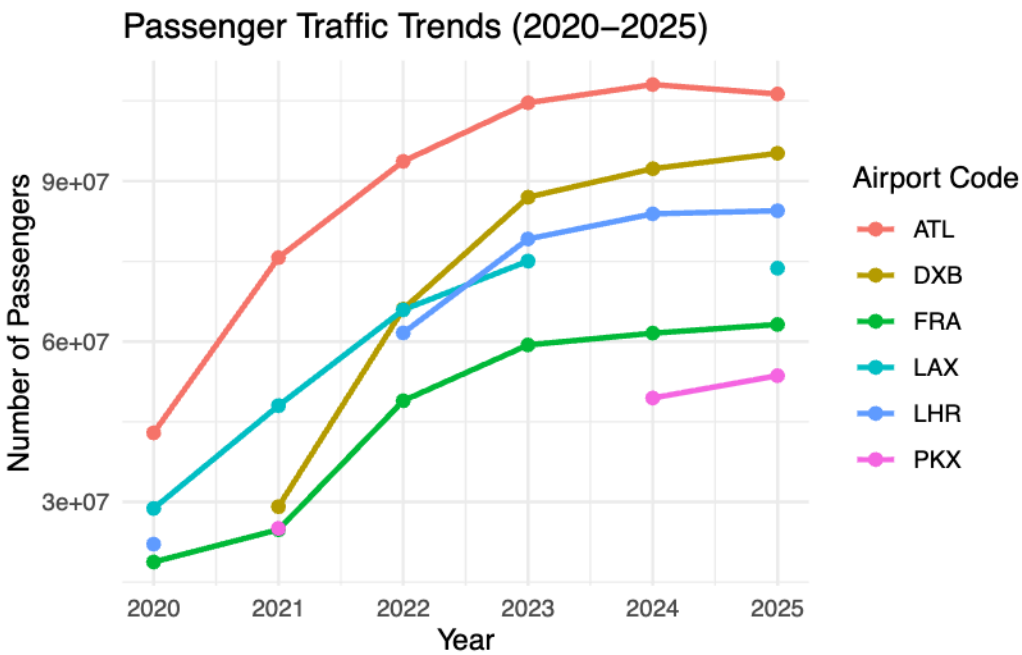

Figure 1: Passenger traffic trends for the six busiest airports from 2020 to 2025.

As shown in Figure 1, passenger traffic generally increases across most airports from 2020 to 2025. For example, ATL consistently has the highest passenger counts and shows steady growth over time. Airports such as DXB and LHR show sharp increases after 2021, indicating recovery following lower values in earlier years. In contrast, some airports (e.g., PKX) have missing values in certain years, which is reflected in gaps or shorter lines in the graph.

Monte Carlo Numerical Integration

Monte Carlo numerical integration is a method used to estimate the value of an integral using random sampling. Instead of solving the integral analytically, random points are generated and used to approximate the area under the curve.

In this assignment, random samples were used to estimate the integral of the Beta(2,2) density on the interval from 0 to 1 with y-bounds from 0 to 1.5. As the sample size increases, the estimate becomes more stable and gets closer to the true value.

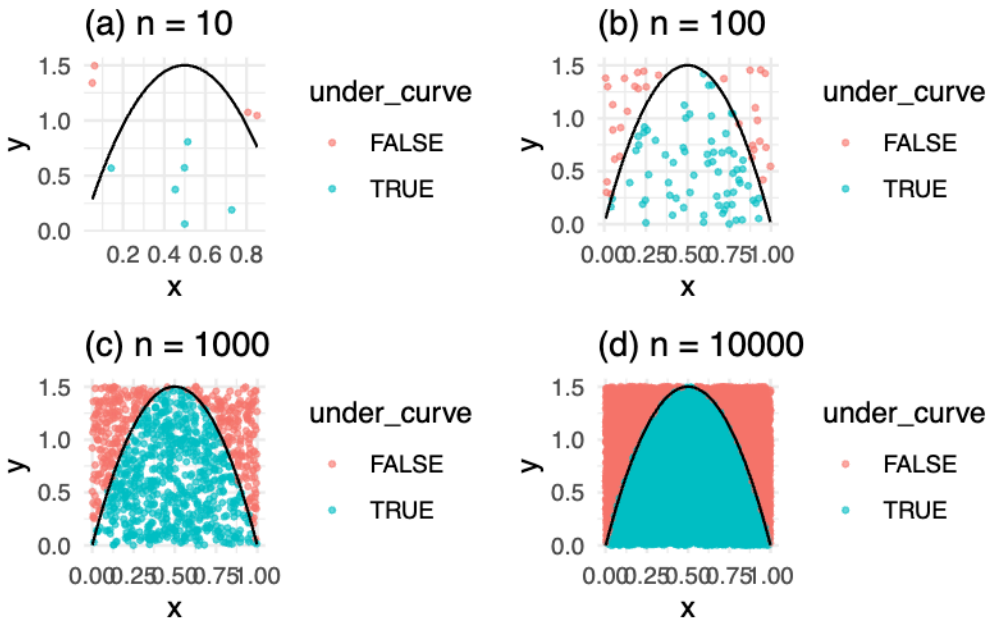

Figure 2: Monte Carlo integration results for different sample sizes (n = 10, 100, 1000, 10000). As sample size increases, the estimate becomes more stable and closer to the true value.

As shown in Figure 2, the small multiple plot shows how the Monte Carlo estimate improves as the number of random points increases. With fewer points, the estimate is rougher, while with more points it becomes more stable and closer to the true value.

Planning and Prompting GenAI Tools

Plan:

My goal was to analyze passenger trends across the six busiest airports from 2020 to 2025.

To achieve this, I followed these steps: 1. Clean and organize the dataset by sorting airports by code and year. 2. Reshape the data into a wide format to create a readable table for comparison across years. 3. Format the table so that passenger counts are clearly displayed and easy to interpret. 4. Create a line graph showing passenger trends over time for each airport. 5. Ensure the graph includes appropriate labels, legend, and title for clarity.

This plan was specifically designed for this dataset, which contains yearly passenger counts for multiple airports, requiring both tabular comparison and time-based visualization.

AI Output:

The AI generated ggplot code that included a line graph with year on the x-axis, passenger counts on the y-axis, and different colors representing each airport. It also suggested adding labels and a legend to improve readability.

Comparison:

The structured prompt produced better results than a more generic prompt because it clearly specified the goal, variables, and chart type. The AI was able to generate usable ggplot code with appropriate mappings and labels.

In contrast, a generic prompt would likely result in a less specific output, requiring additional adjustments such as fixing variable mappings or improving labels. This demonstrates the importance of well-defined prompts when working with genAI tools.

Reflection

In this assignment, I learned how to create a reproducible data analysis document using Quarto by integrating code, outputs, and narrative text into a single organized report. One specific example was creating the table of busiest airports, where I had to reshape the data into a wide format and format the values so that they were readable and fit properly within the page.

I also gained experience using ggplot2 to visualize trends in data. For example, when creating the passenger traffic graph, I initially faced issues with labels being cut off and the plot not fitting properly on the page. Through debugging and adjusting the plot structure, I learned how important layout and labeling are for clear communication.

One key challenge I did face was dealing with the rendering and formatting issues in Quarto.It took me a lot of time for to to adpat and also be really pateint with all the errors ad restaring the sessions . This required me to carefully adjust code and output settings, which improved my problem-solving skills and attention to detail.

Overall, this assignment helped me understand not just how to perform data analysis, but how to present results clearly and reproducibly. I now better appreciate the importance of organizing both code and narrative so that others can easily follow and reproduce my work.

Appendix A: GenAI Usage

Tool Used: ChatGPT (OpenAI GPT-5.3)
Date: April 16, 2026

All responses below are included exactly as generated by the tool without modification.

Prompt 1:

What do I need to include in the Monte Carlo section for HW 4.3 in Quarto?

Response 1 (Full Output):

In the Monte Carlo section, you should include your narrative explanation of the method and your results, along with the final visualization. The code should not be shown in the main document, so you should use a code chunk with echo=FALSE to display only the output (plots). Your explanation should describe how the Monte Carlo simulation works and what insights you observed as the number of samples increases.

Prompt 2:

Should I paste my Monte Carlo code in the main section or not?

Response 2 (Full Output):

No, you should not display your code in the main section of the document. The assignment explicitly states that code should only appear in the Code Appendix. In the main section, you should only show the output (such as plots) and include a clear written explanation. Use echo=FALSE in your code chunks so that only the output is shown.

Prompt 3:

What exactly do I paste for the Monte Carlo code question (function question)?

Response 3 (Full Output):

For the Monte Carlo function question, you need to paste only the function definition that generates random points and determines whether they fall under the curve. You should not include plotting code or extra data wrangling. The function should return a data frame containing the generated x and y values and a logical indicator for whether each point lies under the curve.

Prompt 4:

Help me create a ggplot line chart in R to show passenger trends over time grouped by airport code.

Response 4 (Full Output):

You can create a ggplot line chart using the following structure in R:

ggplot(data, aes(x = year, y = passengers, color = airport_code)) + geom_line() + geom_point() + labs(title = "Passenger Trends Over Time", x = "Year", y = "Number of Passengers", color = "Airport Code") + theme_minimal()

This will generate a line chart showing trends over time for each airport, with different colors representing different airport codes. You can further customize labels and themes as needed.

Appendix B: Code Appendix

Load libraries

```
library(dplyr)
library(tidyr)
library(ggplot2)
library(patchwork)
```

Dataset code

```
busiest_airports <- tribble(
  ~airport, ~code, ~year, ~passengers,
  "Hartsfield-Jackson Atlanta International Airport", "ATL", 2020, 42918685,
  "Hartsfield-Jackson Atlanta International Airport", "ATL", 2021, 75704760,
  "Hartsfield-Jackson Atlanta International Airport", "ATL", 2022, 93699630,
  "Hartsfield-Jackson Atlanta International Airport", "ATL", 2023, 104653451,
  "Hartsfield-Jackson Atlanta International Airport", "ATL", 2024, 108067766,
  "Hartsfield-Jackson Atlanta International Airport", "ATL", 2025, 106302208,

  "Frankfurt Airport", "FRA", 2020, 18768601,
  "Frankfurt Airport", "FRA", 2021, 24812849,
  "Frankfurt Airport", "FRA", 2022, 48918482,
  "Frankfurt Airport", "FRA", 2023, 59355389,
  "Frankfurt Airport", "FRA", 2024, 61564957,
  "Frankfurt Airport", "FRA", 2025, 63189666,

  "Beijing Daxing International Airport", "PKX", 2020, NA,
  "Beijing Daxing International Airport", "PKX", 2021, 25051012,
  "Beijing Daxing International Airport", "PKX", 2022, NA,
  "Beijing Daxing International Airport", "PKX", 2023, NA,
  "Beijing Daxing International Airport", "PKX", 2024, 49441029,
  "Beijing Daxing International Airport", "PKX", 2025, 53618949,

  "Dubai International Airport", "DXB", 2020, NA,
  "Dubai International Airport", "DXB", 2021, 29110609,
  "Dubai International Airport", "DXB", 2022, 66069981,
  "Dubai International Airport", "DXB", 2023, 86994365,
  "Dubai International Airport", "DXB", 2024, 92331506,
```

```
  "Dubai International Airport", "DXB", 2025, 95200000,

  "Los Angeles International Airport", "LAX", 2020, 28779527,
  "Los Angeles International Airport", "LAX", 2021, 48007284,
  "Los Angeles International Airport", "LAX", 2022, 65924298,
  "Los Angeles International Airport", "LAX", 2023, 75050875,
  "Los Angeles International Airport", "LAX", 2024, NA,
  "Los Angeles International Airport", "LAX", 2025, 73709594,

  "Heathrow Airport", "LHR", 2020, 22111469,
  "Heathrow Airport", "LHR", 2021, NA,
  "Heathrow Airport", "LHR", 2022, 61614508,
  "Heathrow Airport", "LHR", 2023, 79183364,
  "Heathrow Airport", "LHR", 2024, 83884572,
  "Heathrow Airport", "LHR", 2025, 84463000
)
```

Table code

```
airport_table <- busiest_airports %>%
  arrange(code, year) %>%
  pivot_wider(
    names_from = year,
    values_from = passengers
  )

airport_table
```

Graph code

```
busiest_airports %>%
  ggplot(aes(x = year, y = passengers, color = code, group = code)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    title = "Passenger Traffic Trends for the Six Busiest Airports (2020-2025)",
    x = "Year",
    y = "Number of Passengers",
```

```
    color = "Airport Code"
  ) +
  theme_minimal()
```

Monte Carlo code

```
f <- function(x) dbeta(x, 2, 2)

generate_plot <- function(n) {
  x <- runif(n, 0, 1)
  y <- runif(n, 0, 1.5)
  under_curve <- y <= f(x)

  df <- data.frame(x, y, under_curve)

  estimate <- mean(df$under_curve) * 1.5

  ggplot(df, aes(x, y, color = under_curve)) +
    geom_point(size = 0.8, alpha = 0.6) +
    stat_function(fun = f, color = "black") +
    ggtitle(paste0("n = ", n, " | Estimate = ", round(estimate, 4))) +
    theme_minimal()
}

plot10 <- generate_plot(10)
plot100 <- generate_plot(100)
plot1000 <- generate_plot(1000)
plot10000 <- generate_plot(10000)

plot10 + plot100 + plot1000 + plot10000
```